Neutrino Selection in ProtoDUNE-HD using Convolutional Neural Networks

Pau Palomar Oliver
Universitat Autònoma de Barcelona
CERN Neutrino Platform (EP-NU)

August 2026

Supervisors: Dario Pullia and Filippo Resnati

Abstract

This report details and evaluates the performance of a Convolutional Neural Network (CNN) model designed to flag neutrino events in the ProtoDUNE-HD Liquid Argon Trime Projection Chamber (LArTPC). We first present the datasets available and the model specific architecture. Then, it is evaluated on Monte Carlo simulations and beam-OFF data, achieving a significant better performance when compare with previous work done making cuts in Pandora. After that, we also apply the model in beam-ON data, where it slightly outperforms the Pandora cuts. Moreover, apart from demonstrating that neutrino events are present in PD-HD, we also expose a discrepancy between expectation and reality in beam-ON data, mainly due to beam-related contamination.

Contents

1 Introduction

The Deep Underground Neutrino Experiment (DUNE) is a long-baseline neutrino experiment hosted by the Fermi National Accelerator Laboratory in the United States. One of the important parts of this experiment is the Far Detector (FD), whose technology has been researched, developed and validated at CERN's Neutrino Platform. One of the FD prototypes that CERN has developed is the ProtoDUNE-HD.

ProtoDUNE-HD (PD-HD) is a horizontal drift Liquid Argon Time Projection Chamber (LArTPC). These prototypes were initially thought to just detect charged particles coming from the SPS beam, to probe and test the technology. However, it was pointed out recently that thanks to the positioning of the detectors with respect to the beam, a neutrino flux could be generated and therefore detected in PD-HD. Thus, by striking the T2 target with 400 GeV protons from the SPS, a set of mesons could be produced, and therefore also neutrinos as a product of their decays. However, the detection of these neutrinos faces a notable challenge. Since PD-HD is on the surface, there is a significant amount of cosmic rays, which require a dedicated trigger and selection process.

In this report, a new machine learning approach is given to these neutrino searches. Previous attempts to select neutrino events were based in applying a set of cuts on reconstructed information from Pandora, while in this project we just rely on image topologies and a machine learning algorithm to select neutrino events and subtract cosmic backgrounds.

2 The ProtoDUNE-HD detector

The PD-HD detector is a LArTPC installed at the CERN Neutrino Platform, and is the continuation of the previous ProtoDUNE Single Phase detector. The PD-HD is placed 720 meters downstream from the T2 target.

The detector is inside a cryostat of dimensions 8.5 m in width and length and 7.9 meters in height. The active volume is divided in two regions by the cathode in the center. In each side, 3.6 m away from the cathode, we find two Anode Plane Assembly units (APAs), with a lenght of 4.6m in total (so 2.3 m each) and 6.1 m in height. An illustration of the detector can be found in Fig. 1. The coordinate system and geometry can be better seen in Fig. 2.

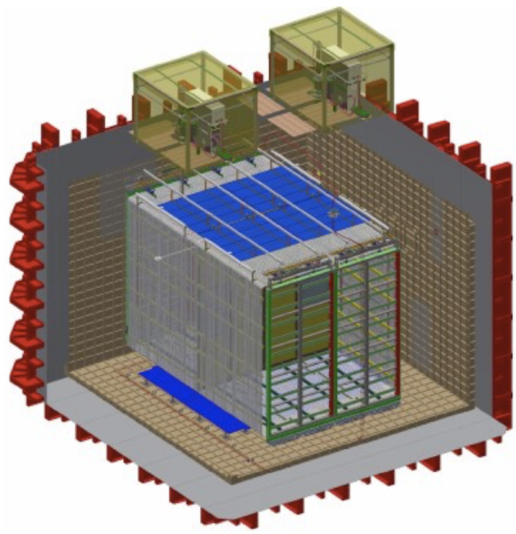


Figure 1: A 3D model of the PD-HD detector.

Each APA is composed of a set of wires that extract signals from the detector. In total there are three wire planes: U and V layers (induction planes) and the Z layer (collection plane). The

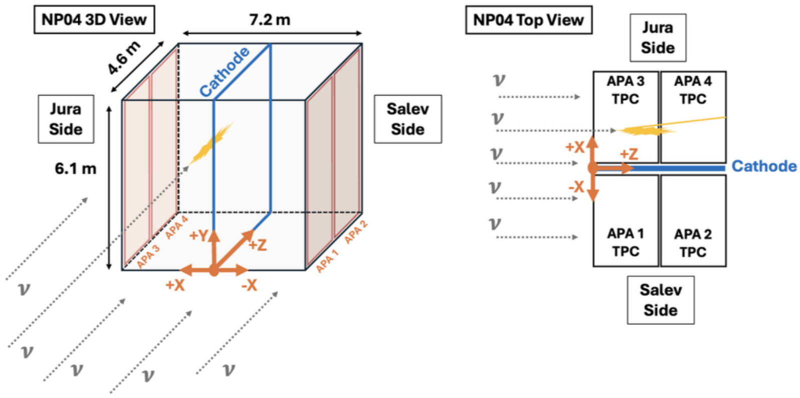


Figure 2: A precise diagram of the PD-HD geometry. Left diagram shows a 3D view of its geometry and coordinate system. In the right a view of the detector from the top can be seen. It is important to note the numbering of the APAs. In this figure the usual numbering that appears in the literature is shown. However, the numbering according to the channel-numbering of the wires should have APAs 2 and 3 labels switched.

induction planes are placed at a specific angle with respect to the diagonal to ensure better event reconstruction. The collection wires, however, are just placed vertically along the APA. A sketch of the APA's wire disposition can be found in Fig. 3. Therefore, for each APA we get three different images (one for each wire plane) of an event. As there are 4 APAs, the full view of the event consists of twelve images.

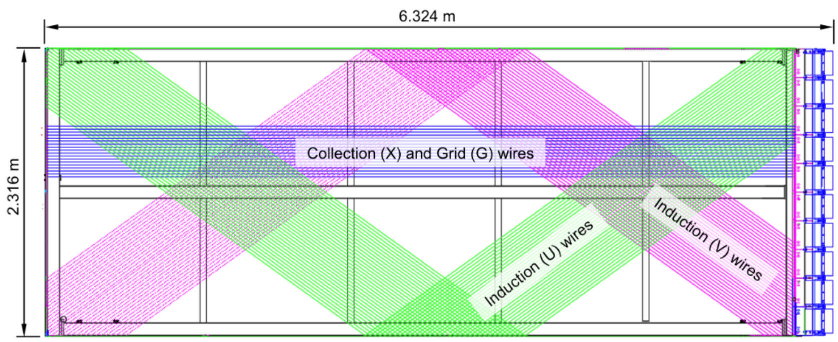


Figure 3: Diagram of a PD-SD APA, which is a very similar version as the ones in PD-HD. The APA is shown turned on its side. In the single phase detector the collection plane was called X, whereas in this report we refer to it as Z. Grid wires are not present in PD-HD.

3 Convolutional Neural Networks

Convolutional Neural Networks (CNNs) have been widely used in various disciplines to analyze images in great detail. One of the most notable examples is early-tumor formation detection, where these models are able to detect tumors at very early stages, completely impossible to be seen by human eye. It is therefore a natural approach to try to apply these models to detect neutrino interactions.

A regular Neural Network (NN) has a 1D array as an input, and after propagating it through a set of layers, it can classify your initial input. For instance, an image of a hand-written number can be fed into a NN, and it can predict with a certain probability which number was written in that image. This is an easy job for a NN, and can achieve great performance. However, since the input has to be flat (i.e. one-dimensional) relevant features of the image can be lost. This is where CNNs come to play: they can handle two-dimensional (technically also three-dimensional) inputs, maintaining those relevant features of the image.

Fundamentally, a CNN scans (more precisely, convolutes) the given image and extracts relevant features through a set of convolutional layers. After that, the output is flattened and fed into a normal fully-connected NN, which analyses the features extracted and classifies the image (or gives any other output as a prediction). For a more detailed explanation of how a CNN works, please refer to [2], as it does not fall in the scope of this report.

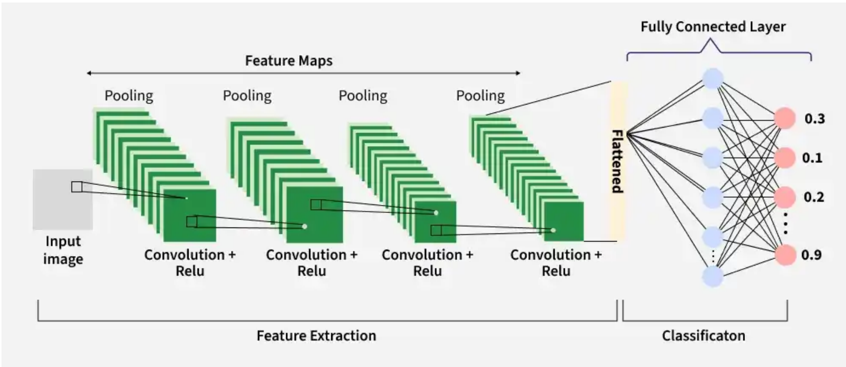
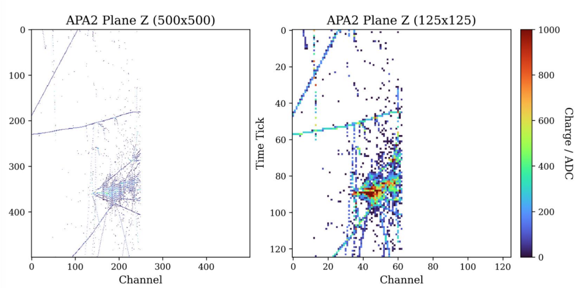


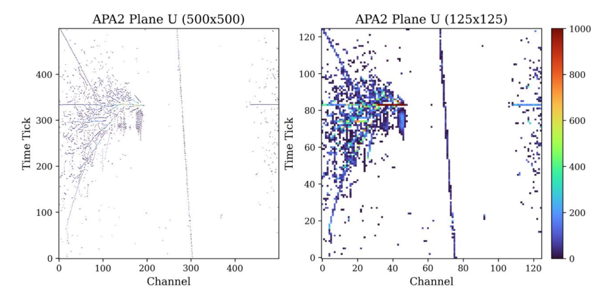
Figure 4: A simple diagram of the main structure of a CNN. It starts with a feature extraction, following with a classification and finally a probability distribution is given as an output.

4 Dataset and Preprocessing

For this project we have access to both Monte Carlo (MC) simulated events and Data from the 2024 run of the PD-HD detector. In particular, the MC dataset has ∼50k events, while beam-OFF data has ∼93k events and beam-ON data has ∼46k events. It is important to note that the MC simulates a wide range of neutrino energies, and we need apply a cut in order to have those of greater energies, which will be the ones that the detector will trigger on. After this selection, 10120 MC events are kept.



(a) Resolution reduction comparison for a MC event in the Z plane of APA 2.



(b) Resolution reduction comparison for a MC event in the U plane of APA 2.

Figure 5: Examples of two different MC events with the original 500x500 image next to the reduced 125x125.

As stated before, each event consists of twelve images. More precisely, 500×500 hit images, which have already been zoomed in the Region of Interest (ROI). Nonetheless, in the first trainings of the model, too much memory usage was needed. In order to solve that problem and reduce runtimes, it was decided to downsample the image resolution to 125×125 (a 16×

improvement in image size). Fig. 5 shows a comparison of this resolution reduction. After this, as expected, runtimes and memory usage substantially improved.

In order to train the CNN model, we need to divide our available data in three sets: training, validation and testing. In this case, 70% was used for training, 15% for validation and 15% for testing. The dataset relies on MC data for the neutrino signal and beam-OFF data for the cosmic ray background (BKG). The objective is to train the model to identify neutrino topologies while rejecting cosmic backgrounds.. Due to the small number of MC events, maintaining a balanced 50/50 sampling across all splits would require discarding approximately 80k BKG events.

Thus, in order to use all the data that we have available, we change the samplings in each set. For the training, we choose a 30/70 MC to BKG sampling, while upweighting and downweighting each class so virtually the model thinks it is a 50/50 sampling. This is done due to the fact that we want the model to see as many BKG events as possible, but we do not want it to be biased by the proportions. For validation and testing this does not matter, and thus we use all BKG events that were not used in training, getting a 4/96 MC to BKG sampling, quite close to the real proportion of neutrino events in the beam-ON data. All of this information is summarized in Table 1.

	Train	**Validation**	**Testing**	**TOTAL**
MC	7084	1518	1518	10120
BKG	16529	36432	36432	89393
Sampling	30/70	4/96	4/96	99513

Table 1: Dataset splitting and sampling ratios for Monte Carlo signal (MC) and cosmic ray background (BKG). The training set is artificially balanced to a 50/50 ratio from a 30/70 sampling to prevent model bias, while validation and testing sets maintain the natural 4/96 close-to-physical distribution.

5 Model Architecture

Now that the dataset splits are defined, we need to construct our model. One of the crucial parts of a model architecture when working with various views for the same image is how will the model see each event. For instance, we could feed the model the 12 different plane views from all 4 APAs independently. The method of inputting data into the convolutional layers is a critical design choice. After trying different ways of doing it, the one that was found to be the best is the following. Images can be grouped in channels, like for instance RGB images, which have one channel per color. Note that channels are correlated, since they share the same shapes. A similar approach can be used in this case, treating each APA as a *colored* image and each plane (U, V and Z) as a different *color*. This makes sense since, analogous to multi-channel imaging, the U, V, and Z planes represent correlated projections of the same 3D interaction volume, and a neutrino event in the collection plane will also appear in the induction plane of that same APA.

So our input events consist of 4 sets of images, one per each APA. As pictured in Fig. 6, each APA is fed to a single Shared Feature Extractor. This block consists of three smaller blocks of 4 layers, which are a convolution layer, followed by a ReLU activation, a batch normalization and finally a MaxPooling. The number of filters per layer increases with depth, going from 16 initially to 32 and 64. Batch normalization is used to take into account the fact that there is a great variety of pixel values, ranging from 0 to the order of thousands. By normalizing, we stabilize the neuron activations and weight calculations.

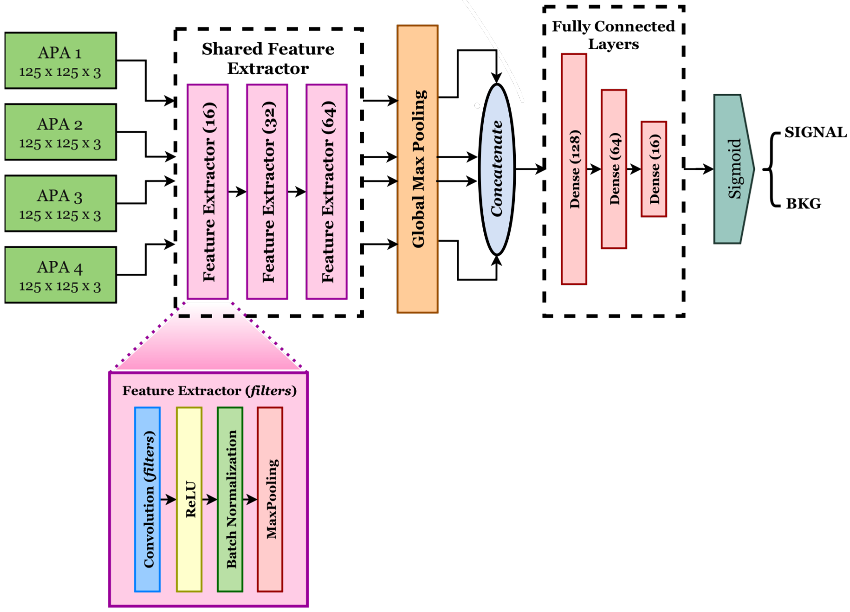


Figure 6: A diagram of the model architecture. The 4 APA's views are fed into a shared feature extractor and passed through a Global Max Pooling layer. Finally, each view is concatenated and the result goes through a set of fully connected layers, which classify the event through the Sigmoid function.

A MaxPooling layer is also present in the feature extractors. MaxPooling is chosen over AveragePooling since we do not want our signal to blur as we go deeper in layers. Since images have lots of empty regions where pixels are set to 0, if we were to average pixels, signal hits would blur and thus we would lose detail in features.

The *Shared* nomenclature essentially means that every APA view gets fed into the same set of layers one by one, thus being affected always by the same weights. For instance, if the weights get adjusted to recognize an electromagnetic shower in APA 1, if then later it also appears in APA 4, the model will be able to recognize it. If we were, however, to have four different feature extractors, one for each APA, the model would be bound to get biased, as it might happen that there is more events with EM showers in APA 2 than in APA 1. By doing this, we make our model *invariant* under APA number.

Moreover, just before concatenation, a Global Max Pooling layer is used. Usually in this stage of the model the outputs are flattened. The primary difference between Flatten and Global Max Pooling is that a Flatten layer essentially takes a $x \times y \times z$ input and outputs a one-dimensional array of length $x \cdot y \cdot z$. After applying several convolutional layers, a significant amount of depth (let's call that the z direction) is accumulated, and thus the resultant length of the array can be very long, implying that lots of parameters will be needed when passed through the fully connected layer. This is where Global Max Pooling comes into play. Instead of directly flattening the array, for every slice of the volume of parameters in the depth direction (i.e. for every z value), it takes the pixel with the biggest value. Thus, the resultant array is not of length $x \cdot y \cdot z$, but of length z.

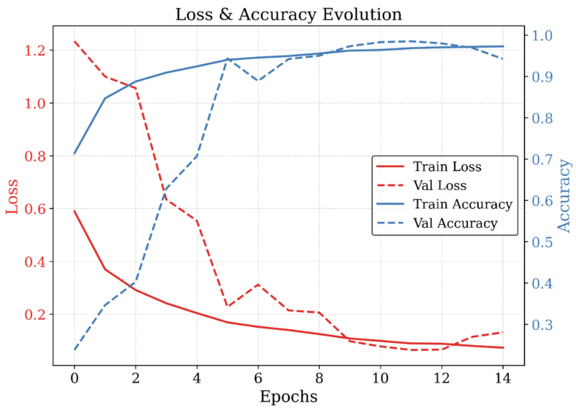
Once the Global Max Pooling is done, all 4 arrays coming from each APA are concatenated and fed to the fully connected layer, which classifies the event. The Sigmoid function assigns a probability between 0 and 1 of it being background (corresponding to 0) or signal (corresponding to 1). In the end, we get a score distribution of all events.

The model uses Binary Crossentropy as its Loss Function, ADAM as the optimizer (with a Learning Rate of 0.001), Early Stopping (with patience 5) and Model Checkpoint as callbacks and it has in the end approximately 66k trainable parameters.

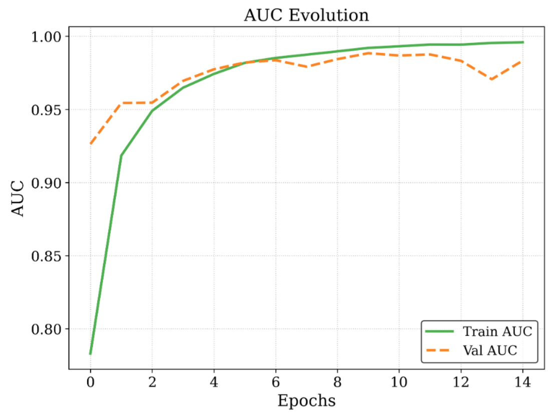
6 Training, Validation and Testing

6.1 Training and Validation

At this point everything is ready to train and test the model. Fig. 7 shows the training metrics in every epoch. Callbacks monitor the AUC metric, since it is robust against class imbalance. Loss and accuracy could be slightly misleading for samplings that are not 50/50. The model weights from epoch 10 are saved as they yield the highest validation AUC. Other metrics are shown in Table 2.



(a) Loss (red) and accuracy (blue) during the training and validation in each epoch. Training corresponds to the solid lines, while validation corresponds to the dashed lines.



(b) Training (solid green line) and validation (yellow dashed line) AUC evolution during all epochs.

Figure 7: Training and validation performance during all epochs.

Metric	**Value**
Total time of execution	0h 29m 4s
Number of epochs completed	15
Average time per epoch	116.3 s
Best Validation Acc	98.46%
Best Validation AUC	0.9884

Table 2: Summary of the CNN training performance and evaluation metrics.

6.2 Threshold Optimization

Before evaluating the model on the test set, we need to set an optimal threshold to the scores that the model assigns to each event. We need to decide what will be labeled as a signal event, and what not. Two optimization criteria are evaluated. First, the threshold can be fixed to achieve a target False Positive Rate (FPR). In this case, the chosen FPR is 0.025%. Moreover, one can also find the threshold that maximizes Poisson Significance (see Eq. 1). Each threshold in the validation set scores is shown in Fig. 8.

$$\chi^2 = 2\left(\mu(\theta) - N_{ON} + N_{ON}\ln\frac{N_{ON}}{\mu(\theta)}\right) \tag{1}$$

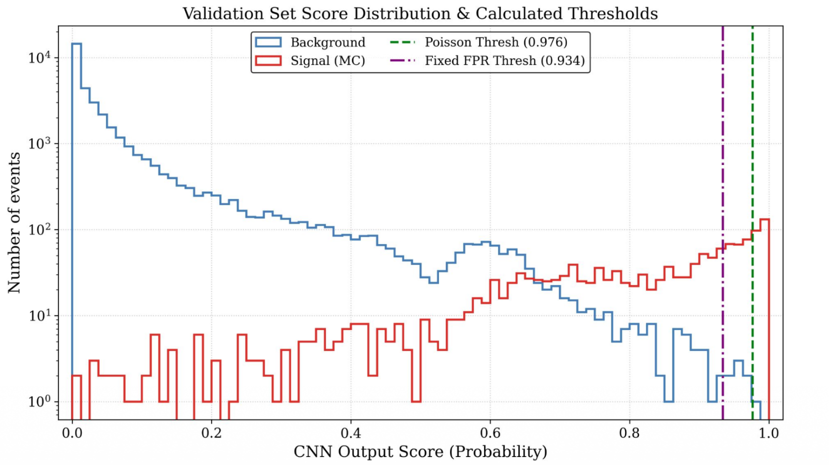


Figure 8: Score distribution of the validation set, in logarithmic scale. Thresholds calculated with both methods (FPR and Poisson Significance) are shown in purple and green.

Thresholds are set at 0.934 and 0.976 for FPR and Poisson Significance, respectively. The FPR-optimized threshold is selected because the Poisson-optimized value is overly strict, leading to a loss in signal efficiency despite its high background rejection.

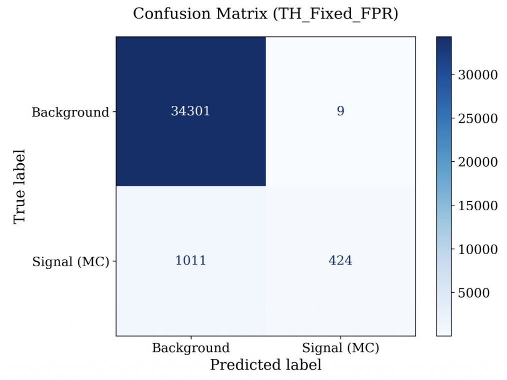
6.3 Testing

After having a threshold set, we can finally apply the model to the test set. It consists of 1435 neutrino events and 34310 BKG events. Table 3 shows relevant testing metrics and in Fig. 9 the score distribution, confusion matrix and ROC curve can be found. These results demonstrate a highly pure signal selection. However, in order to compare them with previous work, we need to rescale the results. That is going to be done in Section 7.1.

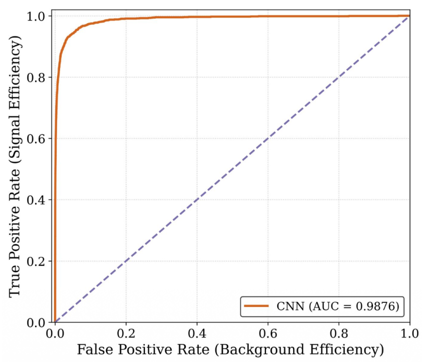
Metric	**Value**
Performance Metrics	
Efficiency (TPR)	29.55%
Background Rejection (TNR)	99.97%
False Positive Rate (FPR)	0.026%
False Negative Rate (FNR)	70.45%
Purity	**97.92%**

Table 3: Testing set CNN evaluation metrics evaluated at a fixed False Positive Rate threshold.

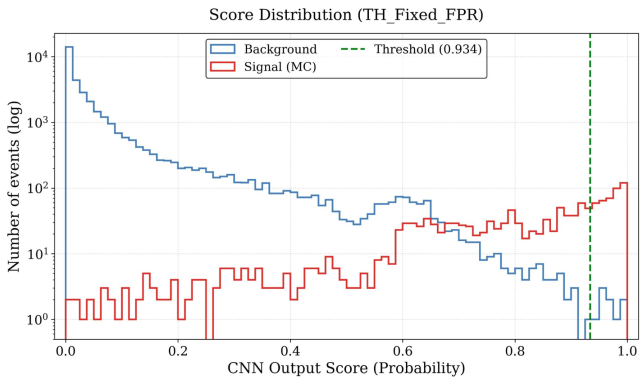
Before comparing results, a fast check in whether the model is biased or not can be easily done. As stated in Section 5, there should not be any preference in APA in the selections. However, when going through some selected events, we find more events in APAs 2 and 4 than in 1 and 3. Fig. 10 shows the simulated flux of neutrino events, and Fig. 11 shows the number of events as a function of the X and Z coordinate. As it can be seen in the Fig. 10, there is more flux in the positive X side of the detector than on the negative. In other words, this means that we have more events in APAs 2 and 4 than in 1 and 3. To make sure that the model is not biased, we can see that in Fig. 11b total events and selected events have approximately the same shape/proportion. Fig. 11a shows the Z coordinate, which is uniform along the detector, just as one would expect.



(a) Confusion Matrix



(b) ROC Curve (AUC = 0.9876)



(c) CNN Output Score Distribution (Threshold = 0.934)

Figure 9: Evaluation of the CNN on the testing dataset. (a) Confusion Matrix displaying absolute event counts. (b) ROC Curve indicating excellent signal-background separation with an AUC of 0.9876. (c) Log-scaled score distribution showing the fixed FPR threshold (0.934) utilized to achieve a high-purity selection.

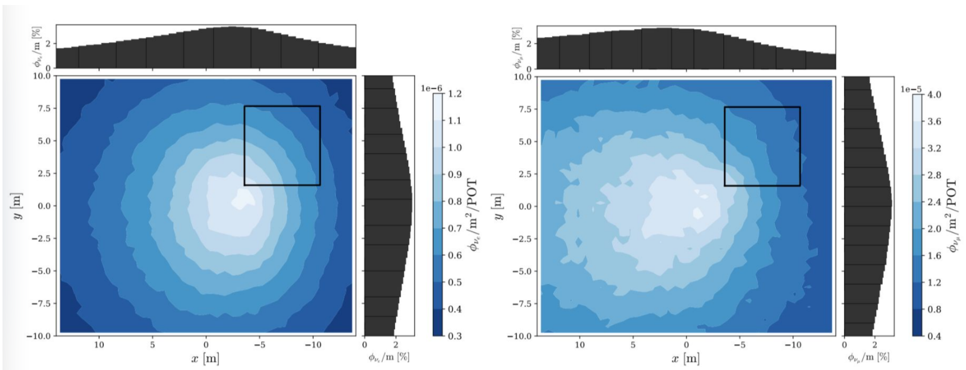
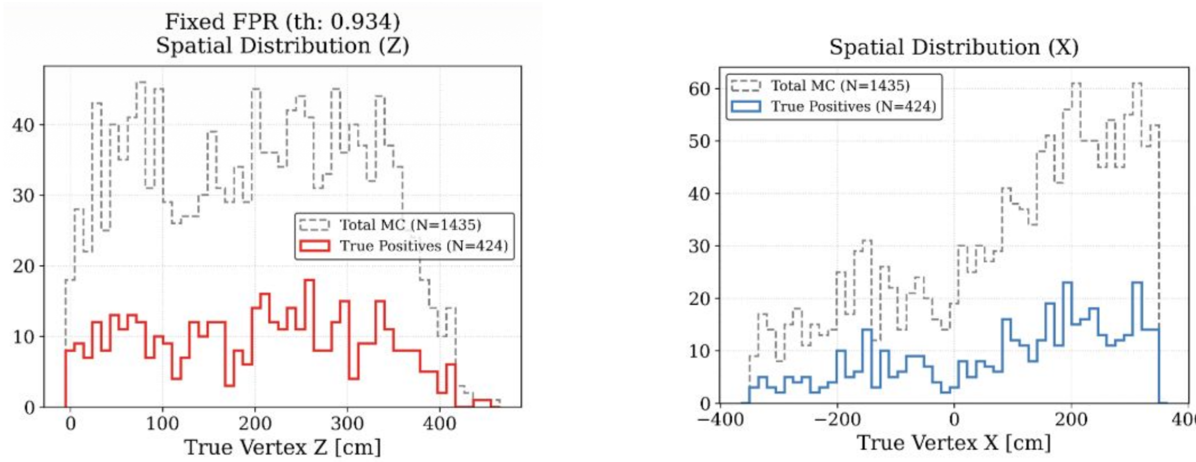


Figure 10: Simulated flux distribution of neutrino events in the XY-plane for ν_e (left) and ν_μ (right) under the wNP04 configuration. The black square represents the projection of PD-HD.

7 Results

7.1 Rescaled expectations on beam-ON data from MC and beam-OFF

As stated before, in order to compare with previous work using classical Pandora cuts, we need to rescale our testing results to what we would expect in the beam-ON data. To do this, we use the total time and Protons On Target (POT) of each dataset. After this is done, we get the results summarized in Table 4, normalized to events per hour. The CNN model yields significant



(a) Vertex Z coordinate distribution in the test set.

(b) Vertex X coordinate distribution in the test set.

Figure 11: Vertex Z (a) and X (b) coordinate truth value from MC events in the test set. The gray dashed line represents all the events in the set, and the solid colored line is the selected events.

improvements across all metrics: signal extraction, background rejection, overall efficiency, and purity.

Selection Method	**Signal (h^{-1})**	**Bkg (h^{-1})**	**Efficiency**	**Purity**
PANDORA	2.85 ± 0.08	0.67 ± 0.12	17.33%	80.97%
CNN Model	**6.85**	**0.20**	**29.58%**	**97.16%**

Table 4: Comparison of expected event rates per hour, selection efficiency, and purity rescaled to beam-ON data between the final classical PANDORA cut and CNN Model.

7.2 Beam-ON data results

Until now, we've just seen the performance of the model when tested with the same type of data that has been trained on. However, we must also test it in a *real* situation, that is, with beam-ON data.

Before going through the results, a first discrepancy already raises. With the rescaling done in last section, we expect our beam-ON dataset to have approximately 34500 events. Nonetheless, as it was stated in section 4, we actually have over 46000 events. These 11500 extra events appear due to beam contamination. As pions decay from the beam (among others, of course, but just to have an example), they produce both neutrinos and muons. Muons are not stopped by the shielding, and therefore reach PD-HD. While most of these muons may not interact in the volume, they deposit more charge than what was expected by just rescaling the MC and beam-OFF data. Therefore, this extra deposited charge makes the trigger from the DAQ of the detector save events more often, and thus why we have that big difference. Of course, some muons also interact in the volume, producing very distinctive topologies: an EM shower with a clear straight previous track. These events have a very similar topology to neutrino events, as both have an EM shower in the same direction. We will comment on the performance of the model in this situation shortly.

After applying the model to the beam-ON dataset, we can plot the score distribution, shown in Fig. 12. It also shows the distribution one would expect when rescaling events from the test.

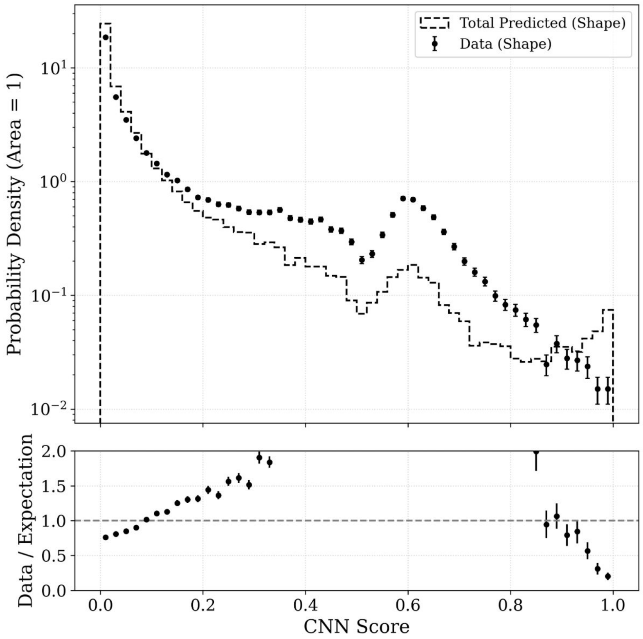


Figure 12: Score distribution of the beam-ON data (black dots) plotted along with the expectations from rescaling the testing data (dashed black line). Plot is normalized and in logarithmic scale in the y-axis. Finally, on the bottom we see a ratio between beam-ON and expectation.

So, if there were no beam contamination, both dashed line and dots would have approximately the same shape. This is indeed the case for low scores, where the model correctly classifies cosmics; it assigns low scores to those events in the beam-ON data that look alike to the ones it trained on. In other words, the model performs the same way in both test and beam-ON at lower scores.

At mid-high scores we can see the model starts to behave differently, as lines diverge. Since these extra events that trigger the detector more often have horizontal trails, the model gets more confused and assigns them scores in the middle range, as it does not know surely how to classify them.

Finally, we see that in the test data we have a small peak in scores very close to 1. This corresponds to the model correctly placing more neutrinos than backgrounds in very high scores (see Fig. 9c). We do not see that in the beam-ON data due to the fact that neutrino events in real data are more noisier and are not exactly the same as the ones in the MC dataset. Thus, the model is less sure and assigns scores slightly lower.

Another way of visualizing the difference in the datasets is shown in Fig. 13. It shows the number of events that pass the selections as a function of the threshold, when the expected number of cosmics (i.e. rescaled backgrounds from testing) are subtracted. As expected, the testing data matches the number of neutrinos at lower thresholds, and starts to lower when the threshold is more strict (since the model is not perfect).

Nonetheless, beam-ON data does not present the same behaviour. When, at lower thresholds, we subtract all the cosmics that we would have expected to see, there still remains a big gap between the amount of neutrinos and the events that are selected. This shows, from another point of view, the fact that we have extra events that we did not account for. As the threshold gets higher, we get rid of these extra events and the model more or less performs the same as it would have been expected (or at least the proportions of events being selected and of those being test-like cosmics are similar).

To end the section, we present a comparison with the results obtained with previous cuts with Pandora. These results are shown in Table 5. We can see that, although mantaining a

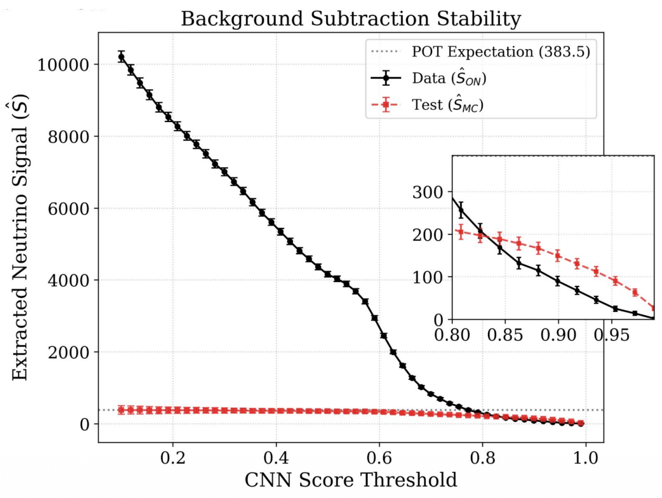


Figure 13: Clean signal with just expected cosmics (rescaled events from beam-OFF) extracted, as a function of the threshold. Grey dashed line marks the expected number of neutrinos in the beam-ON data. Red dots are clean signals from rescaled testing data, and black dots represent extracted clean signals from beam-ON predictions. A zoom in the higher threshold zone is also shown to have more detail.

Selection Method	**Beam-ON events ($\mathbf{h^{-1}}$)**	**Beam-OFF events ($\mathbf{h^{-1}}$)**	**ON/OFF Ratio**
PANDORA	3.63 ± 0.47	0.67 ± 0.12	5.42
CNN Model	**3.45**	**0.20**	**17.24**

Table 5: Comparison of beam-ON and beam-OFF selected event rates per hour and their ratio between the final classical PANDORA cut and CNN Model.

similar result in beam-ON event selection, since the CNN Model has a better cosmic background extraction, results are improved. Nonetheless, one can see in the selected beam-ON events a significant amount of misclasified beam-related muons that interact in the volume, since these have a very similar topology.

8 Conclusions and Future Work

In conclusion, with this project we can clearly see that better results can be obtained by using CNN's instead of classical cuts with Pandora. The model performs significantly better when working with the data that it has been trained on. Nonetheless, it struggles to properly classify neutrino events, as it confuses them with beam-related muons. Therefore, a deeper analysis should be done if one would want to implement this approach as a rigorous classifier for future ProtoDUNE experiments.

Future work to achieve this could be train the model also with beam-ON data, simulating the beam contamination in the Monte Carlo dataset, explore other model architectures and see if performance is better, or even make the model predict other properties of an event, such as the interaction vertex position, energy deposited or neutrino flavor.

References

[1] Abi, B. et al. (DUNE Collaboration). *First results on ProtoDUNE-SP liquid argon time projection chamber performance from a beam test at the CERN Neutrino Platform*. JINST 15 (2020) P12004.

[2] A. Karpathy, J. Johnson, and F. Li. *CS231n: Convolutional Neural Networks for Visual Recognition – Convolutional Neural Networks (CNNs / ConvNets)*. Stanford University. Available at: https://cs231n.github.io/convolutional-networks/#layers (Accessed: August 2026).